TECNOLOGIAS
E ARQUITETURA

How Requirement Gathering influences Technical Debt and Quality in Software Development

Master's in Computer Science and Business Management

Gonçalo Gonçalves Miranda

Supervisor(s):
PhD Maria Cabral Diogo Pinto Albuquerque
Assistant Professor, ISCTE-IUL
PhD Luísa Cristina da Graça Pardal Domingues Miranda
Assistant Professor, ISCTE-IUL

ISCTE – Instituto Universitário de Lisboa
June, 2025

Acknowledgement

Write here the acknowledgments and grants, if any

Abstract

In software engineering, the quality of Requirements Engineering (RE) is a key factor when predicting long-term success and sustainability of software systems. Requirements are the basis for any decision in the project, wether it is related to design, implementation, or maintenance. However, in practice, requirements are often incomplete, ambiguous, or unstable. These conditions provide a breeding ground to what is known as Requirements-Related Technical Debt (RTD). RTD captures the trade-offs made during the requirements phase that lead to architectural, functional, or documentation failures, which lead to increased maintenance costs, reduced adaptability, and long-term quality degradation.

While the general concept of Technical Debt (TD) has been thoroughly studied, the influence of requirement-related factors remains underdeveloped in the particular context of big corporations, where agile and fast-paced development environments are predominant and where decisions made under pressure or with little stakeholder involvement can silently build up as debt, with consequences that may only be detected down the line. Despite existing recommendations to neutralize such problems, the industry still suffers to consistently identify, track, and prevent RTD before it negatively impacts software evolution.

In order to tackle this gap, this research combines a Systematic Literature Review (SLR) of 30 publications with a case study elaborated in cooperation with a software company. The purpose is to consolidates existing work on RTD causes and mitigation approaches, and to validate these findings through empirical investigation. By doing this, the research aims to yield practical insights and theoretical clarity to support more resilient and sustainable RE practices.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

As modern software systems increase in complexity and scale, the difficulty of maintaining stakeholders expectations aligned with system behavior also becomes more difficult and consequential, while also being more critical than ever [1]. Alongside that, both the quality and sustainability of a system's architectures heavily rely on early development life-cycle activities, with a huge focus on the quality of the requirement engineering (RE) process [2]. According to Behutiye et al. [3], the robustness of this process must match the system's complexity, in order to ensure the system remains maintainable, adaptable and aligned with the long-term vision of the quality goals of the project. According both to Behutiye et al. [3] and Melo et al. [4], this means the rigor, clarity, and completeness of the captured requirements influence not only the short-term development process but also the system's ability to progressively adapt to different contexts and new demands. Failing to attentively perform the activities related to this early phase of software engineering increases the likelihood of accumulated architectural and design flaws, paves the way for technical debt (TD).

Although awareness of the risk incomplete, unstable or ambiguous requirements pose to a project is rising, many continue to suffer due to this factor, particularly in fast-paced environments where adaptability are prioritized over documentation and long-term planning, such as contexts related to agile and startups [3, 5]. Though agile methods may be beneficial for swift delivery and user-centric development, they often tend to undermine non-functional requirements (NFRs) and a solid architectural design, which contributes to challenges in the long run, such as the need to refactor certain parts of the system or even quality degradation [6], being the dynamic between short-term delivery goals and long-term software robustness becomes evident in cases where requirements deviate halfway through development or are under-specified.

Research in requirements-related technical debt (RTD) - which is one of the most common types of TD, according to Wattanakriengkrai et al. [7] - unveils that decisions made under uncertainty or pressure tend to output "quick fixes" that accumulate over time and hinder future changes, testing, or scale, as stated by Melo et al. [4]. In theory, these trade-offs are not inherently harmful, as they can be useful tools when correctly managed. However, in practice, RTD is usually left unidentified or undocumented during the requirements phase, resulting in developer teams that are oblivious to the existence of such issues, until they cripple development progress, as documentation regarding quality requirements (QRs) is usually less of a priority for development teams, regardless of the potential problems this is likely to cause in the long-term, according to Behutiye et al. [3].

The risk introduced by poor RE is particularly alarming in organizations that rely on swiftly composed or distributed teams. Due to this, this study aims to explore the influence of requirement gathering practices on the accumulation of TD and the quality of software outputs, while exploring what are the most common causes of RTD and how they can be identified and correctly mitigated in a real world environment.

CHAPTER 2

Theoretical Background

This chapter introduces the core theoretical concepts required to frame the relationship between RE practices and the emergence, accumulation, and management of TD, with a particular emphasis on RTD. It clarifies what is meant by requirements (and their main categories), how RE is commonly understood in contemporary software development contexts, and how TD is conceptualized in the literature, including its different types and mechanisms of propagation. The chapter concludes by positioning requirements quality as an upstream driver of downstream debt, linking foundational definitions to the phenomena explored in the Systematic Literature Review (SLR).

2.1. Requirements and Types of Requirements

In software engineering, requirements constitute the primary means by which stakeholder intent is transformed into explicit expectations for a software system, guiding both implementation decisions and quality objectives [8, 9]. Rather than being merely a technical specification artifact, requirements operate at the intersection of business concerns and technical realization, bridging communication between organizational stakeholders and development teams [2, 8].

From a conceptual perspective, requirements can be viewed along two complementary dimensions. The first distinguishes business requirements (BRs), from system requirements (SRs) [4, 9]. The second dimension differentiates between functional requirements (FRs) and NFRs [3, 8].

For the purposes of this study, FRs and NFRs are treated as the two primary categories of requirements, while acknowledging that both may originate at either the business or system level [9]. This perspective supports a unified view of RE, in which both value-oriented concerns and technical constraints are considered [8], and provides a foundation for analyzing how deficiencies in requirement quality contribute to the emergence and accumulation of TD [2, 4].

2.1.1. Quality Attributes and Quality Requirements

quality attributes (QAs) describe the non-functional characteristics that determine how a software system performs, evolves, and sustains value over time. Commonly cited QAs include performance, reliability, security, maintainability, usability, and scalability, among others. In RE, these attributes are typically captured through NFRs or, more specifically, through QRs, which constrain and shape the realization of functional behavior [3, 8].

Unlike FRs, which define discrete system capabilities, QAs are inherently cross-cutting and often span multiple system components and development activities. As a result, they are more difficult to specify, validate, and prioritize, particularly in agile and iterative development contexts [9]. When quality attributes are insufficiently specified, deferred, or implicitly assumed,

their realization is frequently postponed to later development stages, increasing the likelihood of architectural erosion, rework, and the accumulation of TD [2, 4].

For this reason, QAs play a central role in this study's analysis of requirement quality. Their treatment provides an essential lens for understanding how shortcomings in requirements engineering practices translate into long-term software quality degradation and requirements-related technical debt.

2.1.2. Business and System Requirements

In addition to the commonly adopted distinction between FRs and NFRs, current literature implicitly recognizes a layered view of requirements based on their level of abstraction and purpose. In this view, requirements can be broadly categorized as BRs and SRs. This categorization is particularly relevant in the context of RE, as decisions and omissions at both levels can independently or collectively contribute to the accumulation of RTD [4, 10].

BRs capture the high-level goals, constraints, and value propositions that motivate the development of a software system. They express stakeholders expectations regarding the system, framing success in terms of organizational objectives, user needs, and strategic outcomes. Within agile and industrial development contexts, BRs are frequently gathered through stakeholder interaction, informal documentation, and evolving prioritization mechanisms such as backlogs or roadmaps [3, 11]. When BRs are unclear, unstable, or weakly aligned across stakeholders, they create uncertainty that propagates downstream, increasing the likelihood of misaligned implementations and corrective rework [4, 12].

SRs operationalize business intent by specifying how the software system should behave and which constraints it must satisfy. They translate business goals into concrete, verifiable expectations that guide architecture, design, implementation, and testing activities. In the reviewed literature, SRs are closely associated with both FRs and NFRs, as they define not only system functionality but also QAs such as performance, security, reliability, and maintainability [9, 13]. Deficiencies at this level—such as incomplete specifications, undocumented assumptions, or insufficient consideration of QRs—often force development teams to make implicit decisions, which are a well-documented source of TD [4, 14].

BRs and SRs are therefore understood as complementary layers within RE. Together, they establish the context and constraints under which technical decisions are made. [3, 8].

2.1.3. Functional and Non-Functional Requirements

Software requirements capture both the intended functionality of a system and the quality constraints that shape its behavior and long-term evolution. These dimensions are, however, not independent, as decisions made at the level of the functional scope often entail architectural, organizational, and coordination-related consequences, particularly in agile and large-scale development contexts [15, 16]. Similarly, non-functional constraints such as maintainability, security, and performance directly influence how functionality can be implemented, evolved, and sustained over time, and when left implicit or under-specified, they frequently manifest as long-term TD [17]. Current literature shows that in iterative development settings, the way FRs and

NFRs are specified, prioritized, and refined plays a central role in determining software quality (SQ) outcomes and the accumulation of TD, as feature-driven prioritization often defers quality concerns unless they are explicitly embedded in the requirements process [15, 17]. For this reason, FRs and NFRs are treated in this work as complementary components of the requirements landscape, whose combined management is essential for mitigating TD and improving long-term SQ.

FRs express what the system should do, typically describing services, behaviors, and interactions. Within agile and iterative contexts, FRs are frequently captured as backlog items, user stories, or incrementally refined specifications, and tend to be prioritized due to their immediate visibility and their direct relation to delivered functionality [3, 9]. While this prioritization supports short-term delivery, it also produces an imbalance when FRs evolve without corresponding updates to architectural assumptions, documentation, or verification assets, increasing the likelihood of debt accumulation [2, 4].

NFRs describe how the system should behave and which QAs it must satisfy, such as performance, security, reliability, maintainability, and usability. Current literature often discuesses NFRs in close association with the notion of QRs, reflecting their role in constraining design decisions and shaping long-term system properties [3, 9]. Empirical evidence indicates that QRs are frequently under-documented or treated implicitly in fast-paced development environments, particularly in agile settings, which increases the likelihood of later refactors and architectural degradation [3, 14].

Requirements quality is not treated as an abstract ideal but as a practical determinant of downstream development and usage outcomes. Femmer and Vogelsang [8] emphasizes that requirements quality directly influences quality-in-use, implying that unclear or incomplete requirements are systematically translated into avoidable defects, refactors, and degraded user and stakeholder outcomes, meaning that low-quality requirements increase the probability of negative downstream effects, while high-quality requirements act as a preventive control by reducing ambiguity and support consistent implementation and architectural decisions over time [4, 8].

2.2. Technical Debt

TD is generally conceptualized as the accumulation of deferred work or suboptimal decisions that provide short-term benefit but impose future cost. In agile software projects, TD is frequently linked to pressure-driven trade-offs, incomplete knowledge, and prioritization mechanisms that favor immediate delivery [2, 18]. Current literature also emphasizes that TD is not limited to code-level issues: it can emerge across artifacts, practices, and organizational structures, producing compounding effects on long-term maintainability and team productivity [4, 15].

CHAPTER 3

Related Work

3.1. Systematic Literature Review

In order to summarize all current relevant academic knowledge regarding the topic subject to research, to identify research gaps and to indicate new areas for posterior investigation, this study was conducted as a SLR and followed the guidelines defined by Kitchenham, Charters, et al. [19], that divides the research into 3 different stages: the review planning, the review conduction, and the review report. As per the authors, a systematic approach is predefined using a review protocol that creates research questions and the methods used in the review, in order to target such questions. This allows for the study to be repeatable, which improves its transparency and enables any reader to assess a study's rigor and integrity [19].

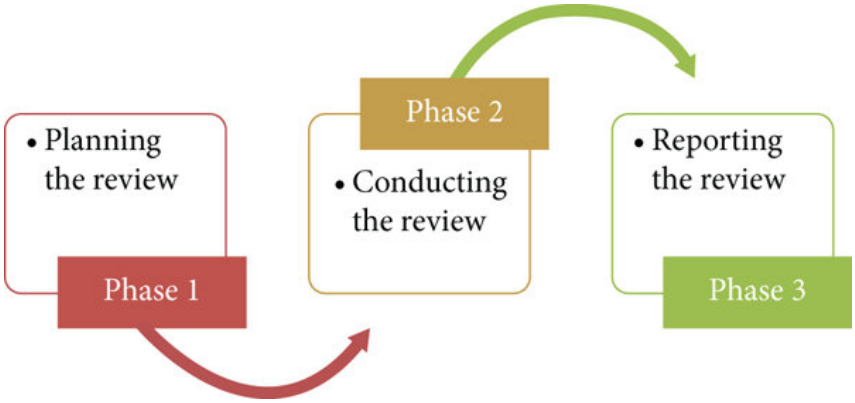


Figure 1. SLR Phases

3.1.1. Review Planning Stage

This section refers to the SLR methodology's first stage, where the rationale for conducting the review should be outlined, as well as the research questions, and where the review protocol is defined.

3.1.1.1. Review Motivation

Despite growing awareness of TD in software engineering, the impact of RE on its mitigation and accumulation remains underexplored. Unreliable requirements are often mentions as key actors to RTD, mainly in environments where documentation and QRs are often de-prioritized, such as agile and fast-paced development environments [3]. Melo et al. [4] states that, while causes and metrics for RTD have been already draft by some studies, current work lacks a systematic and comprehensive compilation of how these issues are generated, as well as their impact on long-lasting product quality, and how they can be addressed correctly. Case studies

that concern both small and large projects reveal that poor requirement gathering techniques can lead to extensive refactoring due to product misalignment with client expectations and to increased maintenance costs, indicators that are highly correlated with TD [5]. This SLR is produced based on the need to clarify the connection between deficient RE and TD, to understand their long-term effects on SQ, and to identify ways to predict and mitigate RTD in heterogeneous development contexts.

3.1.1.2. Research Questions

The focus of this review is to address the following questions:

RQ.1: How do incomplete or evolving requirements contribute to the accumulation of TD in software development projects?

RQ.2: What is the relationship between the quality of requirement gathering practices and software product quality outcomes over time?

RQ.3: Which requirement-related factors most commonly lead to TD, and how are they identified and mitigated in practice?

RQ.4: What RE process is most commonly linked to TD, and how can they improve to mitigate it?

RQ.5: How do different project contexts and team structures affect the emergence or management of RTD?

RQ.6: How is the quality of business and system requirements related to technical debt?

3.1.1.3. Review Protocol

The databases considered for the search are SCOPUS[1], the ACM Digital Library[2], and the IEEE Xplore Digital Library[3] .

3.1.2. Review Conducting Stage

This section represents the second stage in the SLR methodology, where the protocol's application is described. The analysis of the extracted data also occurs during this stage.

3.1.2.1. Research Identification

Starting from this study's research question, the necessary keywords to perform the research were extracted in order to generate an insightful search string, as shown in **Table 1**.

[1] https://www.scopus.com/
[2] https://dl.acm.org/
[3] https://ieeexplore.ieee.org/Xplore/home.jsp

Keywords	Technical Debt; Requirements; Engineering; Software
Search String	*("Requirements Technical Debt") AND ("Requirements Engineering") AND "Software"*

Table 1. SLR keywords and search string

3.1.2.2. Study Selection

For this study, 7 filters where used to thin down the search result, listed on **Table 2**.

ID	Filter
1	Title, Abstract or Keywords
2	2018-2026
3	English
4	Articles, Conference Papers, Short Papers, Reviews and Conference Reviews
5	15 or more Citations (applied only ot papers published before 2025)
6	Manual
7	Backward Snowballing Search

Table 2. Search Filters

The research process was commenced with an initial, filter-less search of the selected search string across the 3 selected databases, which provided an output of 3111 results. Then, the first filter was introduced, "Title, Abstract or Keywords," which enabled a reduction in the number of studies to 452, followed by the application of the filter "2018-2026," that reduced the number of studies to be considered to 364. Afterwards, the third filter, "English", was applied, subtracting 5 more studies from the current pool, bringing it down to 359. After this, the "Articles, Conference Papers, Short Papers, Reviews and Conference Reviews" filter was applied, thinning down the possible studies to 356. However, the biggest decrease in number of articles was introduced by the fifth filter, "15 or more Citations (applied only ot papers published before 2025)", that downsized the study pool to only 72. Subsequently, a manual reading of all the studies allowed to both filter duplicate results and to select only relevant studies. With this, there was a reduction of 8 more studies, bringing the number of selected studies down to 64. Finally, this reading also enabled the application of the seventh filter: the "Backward Snowballing Search", which was based on the relevance of the studies and their citations. Only studies that met prior filters and acceptance criteria were selected, resulting in a definitive pool size of 69 articles, which were considered the most relevant studies, with this research's goal in mind. **Table 3** and **Figure 2**, synthesize the process of the selection of the studies.

Database	No Filter	Filter 1	Filter 2	Filter 3	Filter 4	Filter 5	Filter 6	Filter 7
SCOPUS	1 964	208	158	153	151	40	33	37
IEEE Xplore Digital	93	47	31	31	31	6	6	6
ACM Digital Library	1 054	197	175	175	174	26	25	26
Total	3 111	452	364	359	356	72	64	69

Table 3. SLR Filtration Process

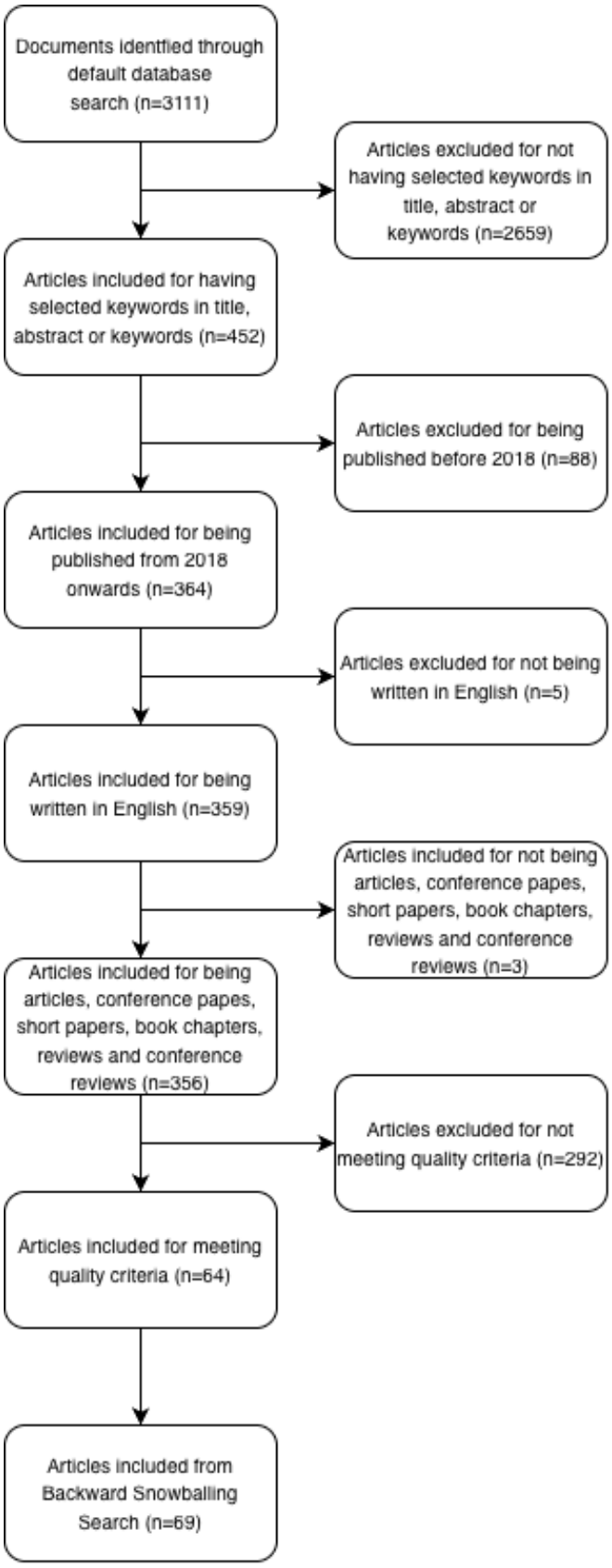
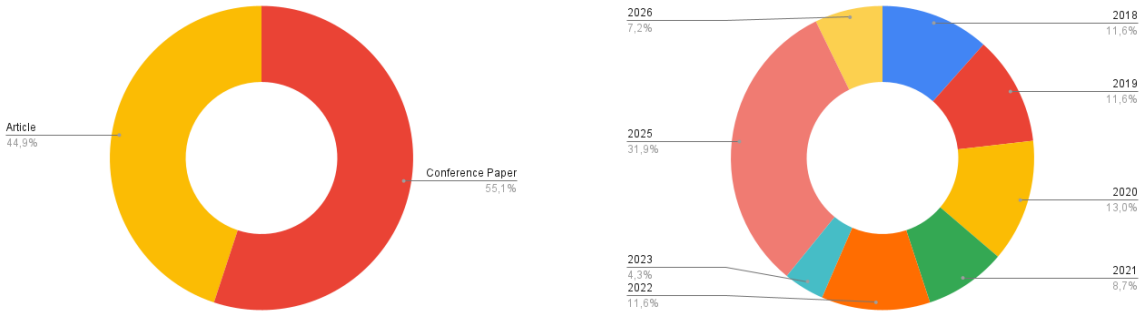


Figure 2. Flow of the Filtration Process

3.1.2.3. Data Extraction and Analysis

Across the 69 studies selected, most were conference papers (55.1%), while 44.9% of the selected studies were generic articles, as shown in **Figure 3**. As for the distribution of the studies by publication year, the most predominant year taken into consideration was 2025, with a significance of 31.9%, 2020 being the second most relevant year in this research (13%). This increase in publications in 2025 may be attributed to a growing research focus on the long-term consequences of RE-related decisions in software systems that have matured under agile and rapid development paradigms. As many such systems reached higher levels of complexity and longevity, issues related to incomplete, evolving, or poorly governed requirements became more visible, motivating renewed empirical attention to the relationship between RE practices and TD [4, 9].



(a) Distribution of Studies by Content Type

(b) Distribution of Studies by Year

Figure 3. Statistics

3.1.3. Review Reporting Stage

This section regards the final phase of the SLR methodology. It encapsulates the extracted data and answers to the outlined research questions.

3.1.3.1. Data Summary

To improve the current understanding on the influence of multiple factors of requirements gathering in TD and SQ, a collection of key terms utilized in the literature to describe a multitude of concepts regarding RTD was determined across the 70 studies. **Table 4** summarizes all the main key term, as well as the number of studies in which each one is used, and the corresponding reference identifiers to those usages. This analysis aims to emphasize both the relevance of certain concepts and the diversity of perspectives and terminologies described across multiple studies.

However, it was also possible to identify several recurring themes regarding the role of requirements gathering and its connection to TD and SQ. These were then grouped into four main topics: the Relation of RE with SQ and TD, the Requirement Volatility and Incompleteness as Debt Factors, the agile, Startups, and Fast-Paced Development Environments, and the Documentation, Traceability, and NFRs. **Table 5** displays both the number of studies that address each one of the previously mentioned topics and their respective references.

Table 4. References by Key Terms

Key Term	References by ID	Total
Requirements Engineering	S4, S5, S7, S8, S9, S10, S11, S12, S14, S15, S17, S27, S38, S39, S40, S44, S45, S49, S54, S57, S58, S60, S64, S65, S68	25
Technical Debt	S1, S2, S5, S6, S7, S8, S9, S10, S11, S12, S13, S16, S18, S19, S21, S22, S24, S26, S28, S30, S32, S36, S37, S40, S42, S43, S44, S45, S47, S48, S49, S50, S51, S52, S54, S55, S56, S58, S59, S60, S62, S64, S69	43
Quality Requirements	S5, S10, S16, S20, S35, S37, S39, S57, S58, S60, S65, S68, S69	13
agile	S4, S5, S15, S16, S17, S31, S38, S39, S40, S41, S45, S47, S54, S55, S64, S68	16
Startups	S31, S45, S64, S68	4
Non-Functional Requirements	S5, S10, S16, S20, S30, S35, S37, S39, S50, S54, S56, S58, S60, S65, S68, S69	16
Self-Admitted Technical Debt	S1, S2, S6, S13, S28, S36, S43, S50, S54, S62	10
Maintainability	S10, S16, S18, S20, S35, S36, S37, S40, S42, S43, S44, S45, S48, S51, S52, S54, S58, S60, S62, S68, S69	21
Refactoring	S6, S19, S32, S44, S48, S51, S52, S59, S68	9
Preventive Actions	S19, S29, S40, S47, S55	5
Mitigation Strategies	S10, S19, S21, S32, S37, S40, S42, S44, S45, S47, S48, S49, S50, S51, S52, S54, S55, S59, S60, S62, S68, S69	22
Requirements Technical Debt	S5, S7, S8, S9, S10, S11, S12, S14, S15, S16, S17, S24, S27, S38, S43, S50, S60, S62, S68	19

Table 5. References for Each Main Topic Identified

Main Topic	References by ID	Total
Relation of RE with SQ and TD	S5, S7, S8, S9, S10, S11, S12, S14, S16, S20, S24, S35, S37, S38, S39, S40, S42, S44, S45, S49, S54, S57, S58, S60, S68, S69	26
Requirement Volatility and Incompleteness as Debt Factors	S7, S8, S14, S18, S31, S37, S45, S54, S64, S68, S69	11
agile, Startups, and Fast-Paced Development Environments	S4, S5, S15, S16, S31, S38, S39, S40, S41, S45, S47, S54, S55, S64, S68	15
Documentation, Traceability, and NFRs	S5, S10, S16, S20, S30, S35, S37, S39, S43, S49, S50, S54, S56, S57, S58, S60, S65, S68, S69	19

Finally, there was another common ground between the studies in use: the challenges faced by development teams when managing TD through requirement practices. Five of the main challenges discussed across current literature are: Capturing and Documenting NFRs, Managing Requirements Volatility and Change, Ensuring Alignment Between Stakeholders and Evolving System Goals, Preventing Long-Term Architectural Degradation through RE Practices, and Refactoring and TD Mitigation Strategies. **Table 6** outlines the amount of studies in which each of the aforementioned challenges are cited and provides the respective reference identifiers.

Table 6. Challenges Identified in the Literature

Challenge	**References by ID**	**Total**
Capturing and Documenting NFRs	S5, S10, S16, S20, S30, S35, S37, S39, S50, S54, S56, S58, S60, S65, S68, S69	16
Managing Requirements Volatility and Change	S7, S8, S14, S18, S31, S37, S45, S54, S64, S68, S69	11
Ensuring Alignment Between Stakeholders and Evolving System Goals	S5, S10, S14, S17, S20, S26, S32, S42, S45, S60, S65, S68	12
Preventing Long-Term Architectural Degradation through RE Practices	S7, S8, S9, S10, S11, S12, S16, S37, S40, S41, S44, S47, S54, S58, S60, S68, S69	17
Refactoring and TD Mitigation Strategies	S6, S19, S21, S32, S37, S40, S42, S44, S45, S47, S48, S49, S50, S51, S52, S54, S55, S59, S60, S62, S68, S69	22

These three tables provided a structured overview of themes explored by current literature. As the data extraction phase aims to provide a deeper understanding of how RE practices relate to the genesis and mitigation of TD, the mapping of these dimensions plays a crucial role into providing these insights.

3.1.3.2. Findings Report

RQ.1: How do incomplete or evolving requirements contribute to the accumulation of TD in software development projects?

Literature consistently identifies incomplete or mutable requirements as foundational causes of TD, especially RTD [4, 20]. Throughout the evaluation of current work, it is clear that the end product of projects based on vague, unstable, or weakly-elaborated requirements is often over complicated and flawed at the architecture level.

Both Seyff et al. [12] and Rios et al. [21] suggest that incomplete requirements often tend to induce developers to make assumptions that bypass formal analysis and validation, resulting in fragile implementations that, in the long-term, create the need to perform code refactoring. Teams resorting to faultier design patterns when NFRs like performance, security, or usability are not explicitly mentioned in the documentation is an example of this, as it may lead to scalability or maintainability issues later on, as mentioned by Wattanakriengkrai et al. [7] and

Farias et al. [22], studies in which it is visible how such gaps amplify the likelihood of Self-Admitted Technical Debt (SATD) - that usually takes form of either comments or workaround code.

agile and fast-paced development environments are the main subjects of this issue. As illustrated by Gupta et al. [5], Villamizar et al. [23], and Vogelsang [24], changing requirements usually emerge iteratively, and occasionally without sufficient alignment between technical teams. This leads to a snowball effect, where architectural and design choices are reconsidered without adequate planning, incentivizing a reactive development culture, as these constant shifts disrupt the stability of the system's foundations and architecture, which, as the complexity increases, become more complex to refactor.

The solution's security risk is also increased by the absence of traceability [16], as in critical fields such as security, vulnerabilities and long-term maintenance liabilities may be introduced due to misaligned or undocumented requirements shifts. Additionally, Melo et al. [4] highlights that the deficiency of formalized procedures for handling mutable requirements leads to suboptimal prioritization, which then results in codebase embedded trade-offs.

As mentioned by Behutiye et al. [3], changing requirements require appropriate TD management strategies, as the absence of such approach is likely to compromise system and design integrity. All things considered, the reviewed studies all point to the fact that incomplete and evolving requirements are the perfect conditions for the genesis of RTD, that can only be minimized by adopting correct approaches to the problem, such as robust change management, comprehensive NFRs documentation, and regular requirement validation cycles.

RQ.2: What is the relationship between the quality of requirement gathering practices and software product quality outcomes over time?

Current literature highlights a strong and evident connection between good quality requirement gathering practices and increased software product quality over time [8]. It is suggested that the deployment of methodical, structured RE practices tend to increase a system's robustness, maintainability, and improves coherence with stakeholder expectations.

Behutiye et al. [3] argues that even in settings where documentation is often minimal, such as agile, integrating methodic requirement practices enhances the compliance between delivered features and long-term goals regarding the system architecture, as the amount of misunderstandings and defects can be significantly decreased by applying methods such as the maintenance of lightweight yet complete requirement documentation, stakeholder verification, and version tracking. However, this is not the only author that defends this perspective. In fact, Melo et al. [4] goes as far as to support it with evidences, showing that in-depth and well-validated requirements enhances both cross-team communication and the on-boarding of new team members, while also generating records regarding the decision-making process, which all lead to a decreased number of misaligned implementations, which tend to lead to RTD.

A connection can also be noted between well-defined requirements and system usability, security, and stability. Recurrent redesigns and regression bugs are usually the result of badly

specified requirements - especially NFRs - as these are in assuring long-term system performance [8]. This is also corroborated by Venters et al. [1], that suggests that system sustainability, which is a metric that covers changeability, performance, and longevity, is a direct consequence of good RE practices.

Vogelsang [24] also emphasize the importance of requirement quality, as they found that low-quality RE processes forces teams to develop reactively, meaning that issues are only resolved real-world failures or constraints, which increases maintenance costs and also degrades stakeholder trust and decreases the likelihood of further investment.

Moreover, organizations who deploy strong RE practices are more predisposed to embrace proactive TD management strategies [20]. These organizations also enforce architectural reviews linked to requirement analysis and maintain traceability records linking requirements, tests, and deliveries, leading to a decreased ambiguity and allowing teams to be more effective at refactoring.

At last, current work indicates that RE quality is a deeply tied to SQ. By utilizing proper documentation, stakeholder involvement, traceability, and iterative validation, requirement gathering practices lead development towards more consistent, scalable, and maintainable solutions, while increasing their quality.

RQ.3: Which requirement-related factors most commonly lead to TD, and how are they identified and mitigated in practice?

RTD manifests as an ongoing, multi-dimensional challenge in the literature, largely caused by a composition between structural weaknesses during the RE process and organizational practices that lack emphasis on long-term maintainability [4]. While it is true that RTD may be dependent on multiple variables, that can independently or collectively increase it, current work systematically emphasizes a pattern in the underlying causes, along with practical methods to both identify and mitigate RTD.

One of the main causes of RTD is the lack of proper specification and documentation of NFRs. Teams usually tend to primarily focus functional aspects due to time-critical deadlines, thus overlooking critical quality aspects such as product performance, security, and scalability [3, 22], which has consequences, as the system evolution continues without structural architecture planning required to fulfill these expectations, meaning that deliveries end up architecturally fragile and employing quick fixes that reflect accumulated debt rather than intentional design.

An aggravating factor to this problematic is the absence of traceability and useful documentation, as mentioned by Rios et al. [21] and Wattanakriengkrai et al. [25]. According to the authors, the decentralization of requirement sources, varying from informal meetings to dispersed documentation tools, is a pivotal element in decreasing consistency, as teams cannot confidently asses the impact certain implementation might have when changes or the reasoning

behind certain decisions are not recorded. This leads to the build up of uncertainty, that subsequently encourages design decisions that emphasize short-term results rather than architectural stability.

The nonalignment between business stakeholders and development teams also leads to the accumulation of RTD; current work shows that the failure to establish communication between the aforementioned actors tends to result in divergent understandings of what a requirement includes. Business needs are prone to swiftly evolving or shift priorities, but without a shared vision, empowered by structured debates or shared artifacts, development teams can utilize outdated or unfinished requirements, resulting in misaligned implementations that are noticeable after their execution, when the cost of refactoring is increasingly bigger than the cost of correctly implementing the correct approach from the get go [11, 12].

Another regularly verified problem is the pace at which requirements progress without adequate architectural revision. Although iterative and agile methodologies embrace change, current work alerts for the risk that constant iteration, without well-planned refactoring, poses, as it inputs systemic misalignment between the initial design and its implementation. This is demonstrated by Herath et al. [16] in respect to security-related features, and how not updating threat models or re-validating architectural assumptions when trying to adapt to evolving business needs compromised projects. Similarly, Villamizar et al. [23] proposes that requirement volatility requires the presence of thorough change management, in order to prevent the dissemination of underlying debt across related modules.

The deviation between RE practices and subsequent technical processes also contributes to the accumulation of RTD. When requirement specification documents are not seamlessly unified with design, testing, and deployment streams, an increase on deficiencies is noticed, yet, they are not flagged until late stages of the development lifecycle. According to Venters et al. [1], the absence of alignment previously mentioned obstructs the organization from a holistic perspective on system quality, constraining their capacity to preemptively address problems.

Current literature suggests a range of mitigation and identification strategies regarding these challenges. Examples of this are the use of automated tools to detect SATD within codebases by analyzing comments and commit messages that may display signs of requirement deviations [26] or the deployment of frameworks dedicated to assess the requirement health, such as those by mentioned by Lenarduzzi and Fucci [10], which present metrics to evaluate the completeness, clarity, and stability of requirement artifacts. Another possible strategy is the establishment of requirement change tracking and control mechanisms, which enable early identification of requirement violations and support more accurate impact analysis when changes are introduced, thereby reducing the propagation of TD [27].

Integrating mitigation approaches in development team culture and workflows is beneficial, as performing frequent stakeholder meetings, recursive refinement of requirement documents, and the inclusion of RE reviews in sprint retrospectives all lead to less RTD build-up. Rios et al. [21] suggests that the integration of RE practices into agile ceremonies, such as backlog

refinements and sprint reviews, helps on the early recognition and remediation of requirements-related issues that impact product quality. Finally, existing research supports the utilization of explicit debt trackers, in which identified RTD occurrences must be documented, prioritized, and prepared for rectification [4].

In conclusion, requirement-related variables that contribute to TD are highly correlated with both technical approaches and organizational culture. The incidence and severity of RTD can be decreases by fostering requirement clarity, traceability, and integrating RE with technical processes, and criticality of RTD. Melo et al. [4] emphasizes that ambiguous, poorly specified, untraceable requirement tend to make software deliveries more prone to refactoring and architecture degradation The author also mentions that well-established practices for the governance leads teams to face less long-term quality issues.

RQ.4: What RE process is most commonly linked to TD, and how can they improve to mitigate it?

Across the analysed literature, the accumulation of TD is commonly associated with agile and fast-paced environments, largely due to the pressure associated with these environments and with how requirements are handled under pressure. Behutiye et al. [9] highlights that QRs are usually deprioritized over FRs, which leads to the underdocumentation of QRs, while also states that TD in agile Software Development is often generated by inadequate or missing knowledge of QRs and FRs. Besides this, due to its focus being continuos delivery, agile Software Development is more prone to TD accumulation than traditional software development [2].

When tackling the situation from a processual standpoint, the recurring option is to strengthen RE in agile environments with concrete practices and artifacts that revolve focus on the solution's architecture and QRs, while still not compromising agile's characteristic iterative nature when creating deliverables, as Snoeck and Wautelet [6] argue that TD is often caused by the lack of architectural thinking associated with agile, and propose an integrated method that enables a decrease of TD by engineering the architecture in a logical, consistent and complete manner.

RE processes can be improved by highlighting TD and RTD and by improving the workflow to introduce governance practices, in order to avoid reactive prioritization, as Besker et al. [18] report that TD prioritization is commonly reactively and lacks the usage of specific strategies and that TD items are often managed and prioritized in an alternative backlog. In conclusion, current literature consistently correlates agile/fast-paced contexts to TD through QRs neglect, as well as frail documentation and reactive handling of TD, while the improvements mentioned are the reinforcement of agile RE practices with explicit QRs and architecture practices, as well as the integration of the backlogs in which TD is tracked into the main backlog of the development teams.

RQ.5: How do different project contexts and team structures affect the emergence or management of RTD?

RTD's existence is strongly dependant on the project's technical and social context, especially on the organizational constraints, stakeholder fragmentation, and on the degree of cross-team awareness over dependencies. In complex, industrial settings, the automotive development is, according to Vogelsang [24], extremely influenced by legacy systems and organizational constraints, as well as OEM/supplier relationships, which worsens the coordination and generates challenges regarding RE. Vogelsang [24] also notes that these contexts are favorable for RTD-relevant dependencies to remain imperceptible, reinforcing that a team's structure and the way knowledge is distributed are directly related to how early requirement-linked issues are detected. In multi-stakeholder environments with various artifacts, the validation and verification of exchanged artifacts becomes, according to Waltersdorfer et al. [20], "cumbersome and error-prone". This, of course, increases the risk of mismatches between requirements and real implementation.

However, RTD management is not just a technical problem - there is also a interpersonal and organizational dimention to it - which Melo et al. [4] conclude is not properly addressed. Overall, the literature converges on the idea that larger or more heterogeneous projects, with legacy constraints, multiple stakeholders, and fragmented tooling, are more prone to RTD and are harder to proactively manage. Besides this, having teams with limited awareness of dependencies and weak cross-stakeholder validation also increase the likelyhood of late discovery and costly remediation of RTD.

RQ.6: How is the quality of business and system requirements related to technical debt?

Across current academic knowledge, there is strong and converging evidence that the quality of both BRs and SRs is a primary determinant of the accumulation of TD, particularly RTD. At the business level, low-quality requirements, such as vague strategic goals, unstable priorities, or insufficient articulation expectations, create misalignment between stakeholders and development teams. This misalignment propagates downstream, forcing teams to make premature or poorly justified design and implementation decisions, which later materialize as architectural rework, scope creep, and persistent corrective effort [2, 18]. Studies show that when BRs fail to explicitly capture long-term concerns, especially those related to SQ and QAs, teams systematically favor short-term delivery over sustainable system evolution, leading to systematic generation of TD early in the lifecycle [27].

At the system level, poor-quality requirements, characterized by ambiguity, incompleteness, volatility, or inadequate documentation, directly translate into implementation-level debt. In particular, under-specified or deprioritized NFRs such as maintainability, performance, security, or reliability are repeatedly identified as dominant sources of RTD, as they constrain architectural decisions only implicitly and are often ignored until late development stages [2, 4]. This deferral results in increased refactoring effort, architectural degradation, and reduced evolvability, reinforcing the accumulation of interest over time. Case studies in large-scale, industrial and agile environments further confirm that insufficient system-level requirements

weaken shared understanding and early decision-making, allowing debt to propagate beyond requirements into architecture, code, documentation, and even process-related artifacts [2, 18].

Furthermore, the literature highlights that deficient requirements are not isolated technical issues but socio-technical ones. Weak alignment between BRs and BRs, combined with ineffective stakeholder communication, amplifies TD accumulation by obscuring responsibility, delaying corrective action, and normalizing workarounds, or ”quick fixes”, as standard practice [28]. From a quality perspective, low-quality requirements structurally undermine SQ, making defects, rework, and corrective maintenance inevitable rather than exceptional [8]. Overall, the evidence consistently indicates that high-quality RE, characterized by clarity, completeness, stability, traceability, and explicit consideration of QAs across both business and system levels, acts as a primary preventive mechanism against TD, whereas deficiencies at either level significantly increase its likelihood, persistence, and long-term impact.

3.1.4. SLR Conclusions

This SLR investigated the link between RE and TD, particularly focusing on the causes and of effects of RTD. Throughout all 30 analyzed studies, it becomes evident that the quality and management of requirements are key factors when analyzing the success and quality of a software system.

A key insight that can be extracted by this study is that incomplete or ever-changing requirements have a very meaningful impact on the compounding of TD. Regardless of the development philosophy, wether it is agile or a more traditional development context, the failure to solidly identify or iteratively validate requirement is strongly linked to the development of fragile architectures, heavy refactoring costs and to the need for long-term maintenance. The research also implies that reactive development practices, in which there is no update to neither the documentation nor to the architecture when accommodating unforeseen changes, are especially harmful, as they build-up over time and blur the original goals of the system.

More fundamentally, the reviewed studies point to the fact that requirements act as an upstream constraint-setting layer: deficiencies at the requirements level systematically limit architectural and design choices, while also preconditioning the form, location, and remediability of TD throughout the system lifecycle.

Another key factor to mention is the impact that requirement gathering quality has in it comes to the quality of software deliverables. Requirement practices that involve stakeholders, thorough documentation, and traceability, are linked to higher-quality software, as solutions are more likely to be more maintainable, scalable, and user-aligned. Contrarily, informal or fragmented RE processes are heavily linked to miscommunication between developer and business teams, misalignment between deliverables and stakeholder expectations, and improvised development decisions, which are a breeding ground for RTD. Current work supports the argument that investing in good practices of RE early on results in lasting advantages in SQ and cost-efficiency.

From this perspective, TD emerges not as an incidental by-product of development pressure, but as a structurally predictable outcome of low-quality or insufficiently governed requirements, whose effects compound as systems evolve.

This study also unveils requirement-related variables that increase TD: it stresses the risk of overlooking NFRs, undocumented changes, and communication gaps between business and technical standpoints. Both cultural and technical shifts are necessary to address these issues. Processes such as SATD detection, requirement traceability, the involvement of stakeholders in the elaboration of requirement artifacts, and the upkeep of explicit TD repositories represent concrete methods teams can use to abandon reactive TD management in favor of a more proactive approach.

Collectively, the outcomes of this review have both academic value and real world applications. As for professionals, the study aims to unequivocally position RE as more than just a preliminary phase of development. This process must be everlasting and evolve alongside the system. For the academy, the review points out research gaps, such as the limited focus on proactive RTD prevention and the influence of organizational culture on RTD build-up. NFRs and domain-specific contexts, such as safety-critical systems, for instance, are also underdeveloped. Future research should focus on developing and deployed prevention frameworks, researching team and organizational variables, and improving NFRs management to reduce long-term cost and respective debt.

In conclusion, this review reinforces that TD is a controllable outcome of intentional design decisions, rather than an inevitable effect of fast-paced development. By treating requirements with the necessary rigor and strategic attention they require, development teams are enabled to create systems that are not only functional, but also sustainable.

3.2. Multivocal Literature Review

In addition to the SLR, a Multivocal Literature Review (MLR) was also conducted, complementing the findings previously obtained from academic literature with grey literature sources, so that more practical perspectives can also be taken into consideration, as the software engineering industry is in constant evolution, and new practices, tools and frameworks are frequently adopted by teams in this industry [29]. The MLR aims to capture insights that may not still be presented in academic publications (and, therefore, that might have not been captured by the previous SLR), thus providing a more well-rounded view of the relationship between RE and TD and what are the practices utilized in the real world to handle this subject. Another goal of this MLR was to also provide better comprehension on how the industry currently defines TD and how RE processes have been adapted over time to better tackle TD management.

3.2.1. Conducting the Literature Review

In order to conduct the MLR, the search string was tweaked, so that it is more adequate to the search engines used. This can be seen on **Table 7**.

The pieces of grey literature retrieved were filtered by the selection criteria presented on **Table 8**.

("Requirements Technical Debt") AND ("Requirements Engineering") AND "Software"

Table 7. MLR Search string

ID	Filter
1	Content must be relevant for the study research
2	Written in English
3	Quality Assessment (QA)

Table 8. Search Filters

It is important to note that, in order to mitigate the inherent risks of bias and varying quality in grey literature, this study employs the AACODS (Authority, Accuracy, Coverage, Objectivity, Date, and Significance) checklist, which is described in **Table 9**, as a formal quality assessment framework. Specifically, the 'Authority' and 'Objectivity' markers are used to distinguish between evidence-based practitioner insights and purely promotional or commercial content. The application of such systematic filter ensures that the multivocal synthesis is grounded in high-quality, reliable data that complements the peer-reviewed academic literature.

Criterion	**Operationalization (0–2 scoring)**
Authority (A)	0: anonymous / unclear author or organization; 1: identifiable author/organization but limited credibility signals; 2: reputable organization or clearly accountable author/editorial ownership.
Accuracy (Ac)	0: claims not supported / no references / unclear basis; 1: partially supported, limited sourcing; 2: internally consistent, evidence-based framing, definitions aligned with established practice.
Coverage (C)	0: very narrow / superficial; 1: partial coverage (e.g., definition + a few tips); 2: comprehensive coverage (e.g., definition + signs + mitigation / measurement / examples).
Objectivity (O)	0: purely promotional / sales-driven; 1: potential commercial bias but still informative; 2: balanced and educational, minimal promotional intent.
Date (D)	0: clearly outdated for the topic; 1: no publication date or limited recency signals; 2: dated and recent (supporting currency of practices).
Significance (S)	0: weak relevance to TD/RE; 1: indirectly relevant; 2: directly relevant to TD management concepts/practices applicable to this research.

Table 9. AACODS quality assessment rubric (0–2 per criterion).

Each non-academic source is rigorously evaluated against these six criteria to ensure credibility, relevance, and methodological soundness, with the resulting assessment and scoring for all included grey sources summarized in **Table 10**.

Table 10. AACODS quality assessment applied to the selected grey literature sources (0–2 per criterion).

ID	Title	Source Author / Organization	Year	A	Ac	C	O	D	S	Total
G1	What is Tech Debt? Signs & How to Effectively Manage It	Atlassian	n.d.	2	2	2	1	1	2	10
G2	What is Technical Debt in Software Development and How to Manage It?	GeeksforGeeks	n.d.	1	1	2	1	1	2	8
G3	What Is Technical Debt: A Guide	Staff	2024	2	2	2	2	2	2	12
G4	What is technical debt?	GitHub	2024	2	2	1	1	2	2	10
G5	Technical Debt (Tech Debt): A Complete Guide	Confluent	n.d.	2	1	2	1	1	2	9
G6	What is Technical Debt? Examples, Prevention & Best Practices	Mendix	2024	2	1	2	1	2	2	10
G7	Technical Debt | Definition & Guide	Sonarsource	n.d.	2	2	2	1	1	2	10
G8	Technical Debt: How to Identify, Measure, and Manage	Islam	2024	1	1	2	1	2	2	9
G9	How to Measure Technical Debt in Software Development	Jain and Solutions	2025	2	1	2	1	2	2	10
G10	Technical Debt & How To Manage It	Mitton	2023	2	2	2	1	2	2	11

3.2.2. Contribution of Grey Literature

The analysis of grey literature sources complements the findings of the SLR by adding a practitioner-orientedand operational perspective to the discussion of TD. Across the selected sources, TD is consistently framed as a pragmatic consequence of delivery pressure, evolving requirements, and short-term prioritization, a view that aligns with academic characterizations of TD as a result of conscious or implicit trade-offs made under constraints [30, 33, 34]. These practitioner-oriented definitions reinforce the SLR conclusion that TD is not produced at random, but rather a predictable outcome of development practices that emphasize speed and flexibility over long-term quality.

While the SLR provides detailed explanations of how deficiencies in RE, such as volatile requirements, under-documented NFRs, and weak stakeholder alignment, lead to the accumulation of RTD, grey literature tends to address TD at a more general level, rarely distinguishing between requirement-related and other forms of debt. Most sources discuss TD holistically, focusing on observable symptoms and downstream consequences rather than on upstream requirement quality as a root cause [31, 32, 36]. This contrast highlights the complementary role of grey literature and how it reflects how TD is commonly understoodand communicated in industry, even when the underlying requirement-related mechanisms remain implicit.

The main contribution of grey literature lies in its emphasis on actionable management and prevention practices. Several sources explicitly highlight practical strategies such as early identification of TD, prioritization alongside feature development, and the adoption of preventive best practices within development workflows [35, 39]. These recommendations resonate with the SLR findings that proactive TD management is more effective than reactive remediation,

while offering concrete examples of how such management is currently operationalized in industrial settings.

Additionally, the challenge of making TD visible and measurable is addressed, by discussing qualitative and quantitative approaches to identifying and tracking debt, including the use of metrics, indicators, and monitoring practices aimed at supporting decision-making [37, 38]. This operational focus complements academic work that stresses the importance of traceability and documentation, by illustrating how organizations often rely on downstream measurement mechanisms when upstream requirement artifacts are incomplete.

From a methodological standpoint, the application of the AACODS framework makes explicit the distinction between conceptual, educational sources and those offering more specialized or practice-oriented insights. While the SLR remains the primary source of causal explanations linking RE quality to RTD, the reviewed grey literature contributes with experience-driven perspectives on tools, practices, and organizational attitudes toward TD management [34, 36]. In alignment with the conclusions of the SLR, this MLR highlights the importance of high-quality RE as a preventive mechanism against TD, while also demonstrating how organizations cope with the consequences of insufficient requirements through reactive and operational management strategies.

CHAPTER 4

Research Methodology

This study follows a mixed approach based in Design Science Research (DSR), which, according to Brocke et al. [40], is a "a problem-solving paradigm that seeks to enhance human knowledge via the creation of innovative artifacts". However, as the study strives to output both an artifact while also an explanation for a real-world phenomenon, it also integrates Case Study Research (CSR), which can be used "before the design of the artifact... to evaluate some preliminary forms or some parts of the artifact" [41].

4.1. Design Science Research

DSR sets the foundation this dissertation. As described by [40], DSR is "a problem-solving paradigm that seeks to enhance human knowledge via the creation of innovative artifacts", which has as its main goal to "extend the boundaries of human and organizational capabilities by designing new and innovative artifacts" [40].

In order to apply this methodology, the problem must be identified, as well as solution objectives, which will lead to the design and development of the final artifact, which will then be demonstrated, evaluated, and communicated [40], displayed in **Figure 4**.

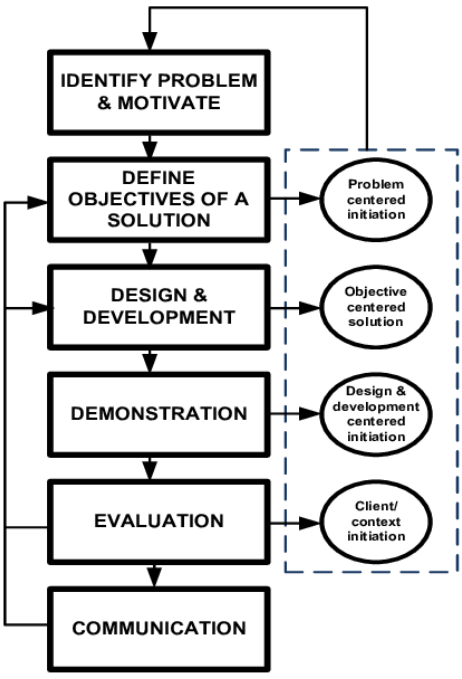


Figure 4. Design Science Research Phases

4.2. Case Study

The study also implements a case study methodology, outlined by Runeson and Höst [42], to explore the impact of RE practices on RTD in a corporate context. The purpose of this case study is to identify and scrutinize how real-world software development teams within the studied company approaches RE and RTD in daily workflows.

The study will collect data by means of interviews, document analysis, and monitoring of project artifacts and sprint ceremonies. Insights will be triangulated in order to comprehend how TD accumulation is impacted by requirement quality, documentation, and traceability. The findings will be evaluated relative to the outcomes of the SLR to validate, contrast, and extend established knowledge. In conclusion, the study seeks to contribute to the generation of relevant, customized suggestions for improving RE practices in the company.

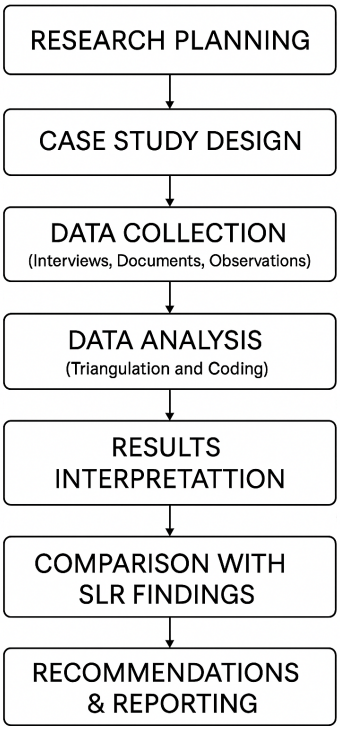


Figure 5. Case Study Elaboration Phases

4.3. Hybrid Methodology

The methodological design of this dissertation combines DSR and CSR. This mixed approach is adopted in order to both analyse an empirical phenomenon and develop an artifact, as Costa et al. [41] emphasizes that CSR provides essential “knowledge about the empirical context”, enabling a better and more precise definition of the problem, stakeholders, and requirements for the artifact in development.

This supports *ex-ante* activities, such as refining objectives and grounding design decisions in real organizational practices.

Nabukenya [43] reiterates the value of utilized an hybrid methodology, arguing that "combining CSR, DSR and AR methods is most suitable to support the execution" of research closely tied organizational practices.

Together, the merger of these two methodologies creates a coherent research strategy that promotes precision, contextual relevance, practical utility and ensures that the most current industry practices are being taken into consideration.

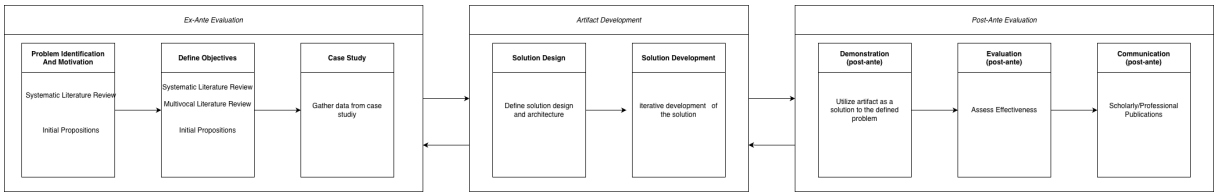


Figure 6. Research Methodology Phases

References

[1] C. C. Venters et al., "Sustainable software engineering: Reflections on advances in research and practice," *Information and Software Technology*, vol. 164, 2023, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2023.107316

[2] N. Rios, M. Mendonça, C. Seaman, and R. Spínola, "Causes and effects of the presence of technical debt in agile software projects," Jun. 2019.

[3] W. Behutiye, P. Rodríguez, M. Oivo, S. Aaramaa, J. Partanen, and A. Abhervé, "Towards optimal quality requirement documentation in agile software development: A multiple case study," *Journal of Systems and Software*, vol. 183, p. 111 112, Jan. 1, 2022, ISSN: 0164-1212. DOI: 10.1016/j.jss.2021.111112 Accessed: Jan. 23, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121221002090

[4] A. Melo, R. Fagundes, V. Lenarduzzi, and W. B. Santos, "Identification and measurement of requirements technical debt in software development: A systematic literature review," *Journal of Systems and Software*, vol. 194, 2022, ISSN: 0164-1212. DOI: 10.1016/j.jss.2022.111483

[5] V. Gupta, J. M. Fernandez-Crehuet, and T. Hanne, "Fostering continuous value proposition innovation through freelancer involvement in software startups: Insights from multiple case studies," *Sustainability (Switzerland)*, vol. 12, no. 21, pp. 1–35, 2020, ISSN: 2071-1050. DOI: 10.3390/su12218922

[6] M. Snoeck and Y. Wautelet, "Agile MERODE: A model-driven software engineering method for user-centric and value-based development," *Software and Systems Modeling*, vol. 21, no. 4, pp. 1469–1494, Aug. 1, 2022, ISSN: 1619-1374. DOI: 10.1007/s10270-022-01015-y Accessed: Jan. 23, 2026. [Online]. Available: https://doi.org/10.1007/s10270-022-01015-y

[7] S. Wattanakriengkrai, R. Maipradit, H. Hata, M. Choetkiertikul, T. Sunetnanta, and K. Matsumoto, *Identifying Design and Requirement Self-Admitted Technical Debt Using N-gram IDF*. Dec. 1, 2018, 7 pp., Pages: 12. DOI: 10.1109/IWESEP.2018.00010

[8] H. Femmer and A. Vogelsang, "Requirements quality is quality in use," *IEEE Software*, vol. 36, no. 3, pp. 83–91, May 2019, ISSN: 1937-4194. DOI: 10.1109/MS.2018.110161823

[9] W. Behutiye et al., "Management of quality requirements in agile and rapid software development: A systematic mapping study," *Information and Software Technology*, vol. 123, p. 106 225, Nov. 1, 2019. DOI: 10.1016/j.infsof.2019.106225

[10] V. Lenarduzzi and D. Fucci, "Towards a holistic definition of requirements debt," in *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, Sep. 2019, pp. 1–5. DOI: 10.1109/ESEM.2019.8870159 Accessed: Jan. 23, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/8870159

[11] C. Gralha, D. Damian, A. Wasserman, M. Goulão, and J. Araújo, "The evolution of requirements practices in software startups," in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, May 2018, pp. 823–833. DOI: 10.1145/3180155.3180158 Accessed: Jan. 23, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/8453156

[12] N. Seyff et al., "Crowd-focused semi-automated requirements engineering for evolution towards sustainability," Jun. 2018. DOI: 10.1109/RE.2018.00-23

[13] M. Alenazi, "The role of environmental assumptions in shaping requirements technical debt," *Applied Sciences*, vol. 15, no. 14, p. 8028, Jan. 2025, ISSN: 2076-3417. DOI: 10.3390/app15148028 Accessed: Jan. 24, 2026. [Online]. Available: https://www.mdpi.com/2076-3417/15/14/8028

[14] E. Scott, G. Robiolo, S. Matalonga, M. Felderer, and D. Pfahl, "A study on reported under-documented non-functional requirements as an indicator of technical debt," *Software Quality Journal*, vol. 33, no. 3, p. 28, Jun. 30, 2025, ISSN: 1573-1367. DOI: 10.1007/s11219-025-09725-4 Accessed: Jan. 24, 2026. [Online]. Available: https://doi.org/10.1007/s11219-025-09725-4

[15] P. Herath and M. O. Ahmad, "A case study on decoding human factors and socio-technical debt in large scale agile project," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25, New York, NY, USA: Association for Computing Machinery, Dec. 23, 2025, pp. 1–9, ISBN: 979-8-4007-1832-8. DOI: 10.1145/3727967.3756846 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3727967.3756846

[16] P. Herath, M. O. Ahmad, and T. Gustavsson, "Product guardian role and socio-technical debt management in large-scale agile," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25, New York, NY, USA: Association for Computing Machinery, Dec. 23, 2025, pp. 127–135, ISBN: 979-8-4007-1832-8. DOI: 10.1145/3727967.3756835 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3727967.3756835

[17] M. Borg, M. Larsson, P. Breid, and N. Hagatulah, "QUPER-MAn: Benchmark-guided target setting for maintainability requirements," in *Proceedings of the 1st International Workshop on Responsible Software Engineering*, ser. ResponsibleSE '25, New York, NY,

USA: Association for Computing Machinery, Jun. 23, 2025, pp. 29–36, ISBN: 979-8-4007-1461-0. DOI: 10.1145/3711919.3728680 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3711919.3728680

[18] T. Besker, A. Martini, and J. Bosch, "Technical debt triage in backlog management," in *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, May 2019, pp. 13–22. DOI: 10.1109/TechDebt.2019.00010 Accessed: Jan. 23, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/8786030

[19] B. Kitchenham, S. Charters, et al., "Guidelines for performing systematic literature reviews in software engineering," 2007.

[20] L. Waltersdorfer, F. Rinker, L. Kathrein, and S. Biffl, "Experiences with technical debt and management strategies in production systems engineering," in *Proceedings of the 3rd International Conference on Technical Debt*, ser. TechDebt '20, New York, NY, USA: Association for Computing Machinery, Sep. 25, 2020, pp. 41–50, ISBN: 978-1-4503-7960-1. DOI: 10.1145/3387906.3388627 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3387906.3388627

[21] N. Rios et al., *Hearing the Voice of Software Practitioners on Causes, Effects, and Practices to Deal with Documentation Debt*. Mar. 18, 2020, 55 pp., Pages: 70, ISBN: 978-3-030-44428-0. DOI: 10.1007/978-3-030-44429-7_4

[22] M. A. d. F. Farias, M. G. d. M. Neto, M. Kalinowski, and R. O. Spínola, "Identifying self-admitted technical debt through code comment analysis with a contextualized vocabulary," *Information and Software Technology*, vol. 121, 2020, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106270

[23] H. Villamizar, M. Kalinowski, M. Viana, and D. Méndez Fernández, "A systematic mapping study on security in agile requirements engineering," Jun. 2018. DOI: 10.1109/SEAA.2018.00080

[24] A. Vogelsang, "Feature dependencies in automotive software systems: Extent, awareness, and refactoring," *Journal of Systems and Software*, vol. 160, p. 110 458, Feb. 2020, ISSN: 01641212. DOI: 10.1016/j.jss.2019.110458 Accessed: Jan. 23, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0164121219302328

[25] S. Wattanakriengkrai et al., "Automatic classifying self-admitted technical debt using n-gram IDF," in *Proc. Asia Pac. Softw. Eng. Conf. APSEC*, Journal Abbreviation: Proc. Asia Pac. Softw. Eng. Conf. APSEC, vol. 2019-December, IEEE Computer Society, 2019, pp. 316–322, ISBN: 978-1-7281-4648-5. DOI: 10.1109/APSEC48747.2019.00050

[26] Y. Li, M. Soliman, and P. Avgeriou, "Identification and remediation of self-admitted technical debt in issue trackers," Aug. 26, 2020. DOI: 10.1109/SEAA51224.2020.00083

[27] S. Freire et al., "Actions and impediments for technical debt prevention: Results from a global family of industrial surveys," in *Proceedings of the 35th Annual ACM Symposium*

on Applied Computing, ser. SAC '20, New York, NY, USA: Association for Computing Machinery, Mar. 30, 2020, pp. 1548–1555, ISBN: 978-1-4503-6866-7. DOI: 10.1145/3341105.3373912 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3341105.3373912

[28] E. Klotins et al., "Exploration of technical debt in start-ups," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18, New York, NY, USA: Association for Computing Machinery, May 27, 2018, pp. 75–84, ISBN: 978-1-4503-5659-6. DOI: 10.1145/3183519.3183539 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3183519.3183539

[29] V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Information and Software Technology*, vol. 106, pp. 101–121, 2019, ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2018.09.006 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584918301939

[30] Atlassian. "What is tech debt? signs & how to effectively manage it — atlassian," Accessed: Dec. 7, 2025. [Online]. Available: https://www.atlassian.com/agile/software-development/technical-debt

[31] GeeksforGeeks. "What is technical debt in software development and how to manage it?" GeeksforGeeks. Section: GBlog, Accessed: Dec. 8, 2025. [Online]. Available: https://www.geeksforgeeks.org/blogs/what-is-technical-debt-in-software-development-and-how-to-manage-it/

[32] C. Staff. "What is technical debt: A guide," Coursera, Accessed: Dec. 8, 2025. [Online]. Available: https://www.coursera.org/articles/technical-debt

[33] GitHub. "What is technical debt?" GitHub, Accessed: Dec. 8, 2025. [Online]. Available: https://github.com/resources/articles/what-is-technical-debt

[34] Confluent. "Technical debt (tech debt): A complete guide," Confluent, Accessed: Dec. 8, 2025. [Online]. Available: https://www.confluent.io/learn/tech-debt/

[35] Mendix. "What is technical debt? examples, prevention & best practices." Section: Blog, Accessed: Dec. 8, 2025. [Online]. Available: https://www.mendix.com/blog/what-is-technical-debt/

[36] Sonarsource. "Technical debt — definition & guide," Accessed: Dec. 8, 2025. [Online]. Available: https://www.sonarsource.com/resources/library/technical-debt/

[37] M. A. Islam. "Technical debt: How to identify, measure, and manage," Innovirtual Software, Accessed: Jan. 25, 2026. [Online]. Available: https://innovirtual.de/technical-debt-how-to-identify-measure-and-manage-it-in-your-codebase/

[38] A. Jain and V. Solutions. “How to measure technical debt in software development,” Visure Solutions, Accessed: Jan. 25, 2026. [Online]. Available: https://visuresolutions.com/alm-guide/measure-technical-debt-in-software-development/

[39] L. Mitton. “Technical debt & how to manage it,” Splunk, Accessed: Jan. 25, 2026. [Online]. Available: https://www.splunk.com/en_us/blog/learn/technical-debt.html

[40] J. v. Brocke, A. Hevner, and A. Maedche, “Introduction to design science research,” in Nov. 2020, pp. 1–13, ISBN: 978-3-030-46780-7. DOI: 10.1007/978-3-030-46781-4_1

[41] E. Costa, A. L. Soares, and J. P. d. Sousa, “Situating case studies within the design science research paradigm: An instantiation for collaborative networks,” in *Collaboration in a Hyperconnected World*, A. Hamideh, L. M. Camarinha-Matos, and L. S. António, Eds., Cham: Springer International Publishing, 2016, pp. 531–544, ISBN: 978-3-319-45390-3. DOI: https://doi.org/10.1007/978-3-319-45390-3_45

[42] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, Apr. 1, 2009, ISSN: 1573-7616. DOI: 10.1007/s10664-008-9102-8 Accessed: Jan. 24, 2026. [Online]. Available: https://doi.org/10.1007/s10664-008-9102-8

[43] J. Nabukenya, “Combining case study, design science and action research methods for effective collaboration engineering research efforts,” pp. 343–352, Nov. 2012. DOI: 10.1109/HICSS.2012.162

[44] M. O. Ahmad, V. Mandić, N. Taušan, A. Katin, and P. Herath, “Technical debt is not just technical: An industrial case study in large agile software development,” *Journal of Systems and Software*, vol. 234, p. 112 719, Apr. 2026, ISSN: 01641212. DOI: 10.1016/j.jss.2025.112719 Accessed: Jan. 24, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0164121225003887

[45] Y. Granados Hondares, M. Snoeck, G. Ferreira Lorenzo, and J. Ruiz de la Peña, “Exploring practitioner’s perception of conceptual modelling in agile software development,” in *The Practice of Enterprise Modeling*, H.-G. Fill, Y. Wautelet, J. Ralyté, and J. Zdravkovic, Eds., Cham: Springer Nature Switzerland, 2026, pp. 141–158, ISBN: 978-3-032-12063-2. DOI: 10.1007/978-3-032-12063-2_10

[46] F. Maiwald, N. Weber, K. Massow, and I. Radusch, “Leaving the tech debt behind: How to sustainably improve the user and developer experience of a legacy frontend by designing, building and migrating to a new web client,” presented at the 21st International Conference on Web Information Systems and Technologies, Jan. 24, 2026, pp. 213–219, ISBN: 978-989-758-772-6. Accessed: Jan. 24, 2026. [Online]. Available: https://www.scitepress.org/Link.aspx?doi=10.5220/0013775500003985

[47] I. Nakamura, Y. Kashiwa, B. Lin, and H. Iida, “Understanding self-admitted technical debt in test code: An empirical study,” *ACM Trans. Softw. Eng. Methodol.*, Jan. 5, 2026,

Just Accepted, ISSN: 1049-331X. DOI: 10.1145/3786791 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3786791

[48] L. Rantala, L. K. Shar, M. V. Mäntylä, W. Minn, and Y. N. Tun, "Studying SATD in drone systems with human-AI collaboration," *Journal of Systems and Software*, vol. 231, p. 112 625, Jan. 1, 2026, ISSN: 0164-1212. DOI: 10.1016/j.jss.2025.112625 Accessed: Jan. 24, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121225002948

[49] A. Tornhill, M. Borg, N. Hagatulah, and E. Söderberg, "ACE: Automated technical debt remediation with validated large language model refactorings," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion '25, New York, NY, USA: Association for Computing Machinery, Jul. 28, 2025, pp. 1318–1324, ISBN: 979-8-4007-1276-0. DOI: 10.1145/3696630.3730565 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3696630.3730565

[50] J.-N. Meckenstock and V. Wallmichrath, "Adapt and overcome - how agile practitioners adapt to issues that impede the delivery of value: An interview study," in *Agile Processes in Software Engineering and Extreme Programming*, S. Peter, M. Kropp, A. Aguiar, C. Anslow, M. I. Lunesu, and A. Pinna, Eds., Cham: Springer Nature Switzerland, 2025, pp. 245–264, ISBN: 978-3-031-94544-1. DOI: 10.1007/978-3-031-94544-1_17

[51] A. Bhatia, F. Khomh, B. Adams, and A. E. Hassan, "An empirical study of self-admitted technical debt in machine learning software," *ACM Trans. Softw. Eng. Methodol.*, Dec. 15, 2025, Just Accepted, ISSN: 1049-331X. DOI: 10.1145/3785001 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3785001

[52] K. Kegel, S. Götz, and U. Aßmann, "Branch drift: A visually explainable metric for consistency monitoring in collaborative software development," *IEEE Access*, vol. 13, pp. 116 972–116 993, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3586537 Accessed: Jan. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11072155

[53] S. Khalfaoui, H. Khalfaoui, and A. Azmani, "Microservices-driven automation in full-stack development: Bridging efficiency and innovation with FSMicroGenerator," *IEEE Access*, vol. 13, pp. 125 131–125 156, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3589285 Accessed: Jan. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11080431

[54] A. Aljohani and H. Do, "PromptDebt: A comprehensive study of technical debt across LLM projects," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '25, New York, NY, USA: Association for Computing Machinery, Dec. 24, 2025, pp. 371–382, ISBN: 979-8-4007-1385-9. DOI: 10.1145/3756681.3756976 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3756681.3756976

[55] M. Alenazi, "Requirements technical debt through the lens of environment assumptions," in *2025 IEEE/ACM International Conference on Technical Debt (TechDebt)*, Apr. 2025, pp. 40–46. DOI: 10.1109/TechDebt66644.2025.00012 Accessed: Jan. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11024540

[56] D. Prajapati and R. Acuña, "Tracing code quality evolution: Insights from a data structures and algorithms course," in *2025 IEEE Frontiers in Education Conference (FIE)*, ISSN: 2377-634X, Nov. 2025, pp. 1–9. DOI: 10.1109/FIE63693.2025.11328661 Accessed: Jan. 24, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11328661

[57] J. P. Biazotto, D. Feitosa, P. Avgeriou, and E. Y. Nakagawa, "Understanding practitioners' reasoning and requirements for efficient tool support in technical debt management," *Empirical Software Engineering*, vol. 30, no. 5, p. 134, Jul. 4, 2025, ISSN: 1573-7616. DOI: 10.1007/s10664-025-10691-5 Accessed: Jan. 24, 2026. [Online]. Available: https://doi.org/10.1007/s10664-025-10691-5

[58] M. Alenazi, "Uncovering the implicit: A comparative evaluation of modeling approaches for environmental assumptions," *Applied Sciences*, vol. 15, no. 19, p. 10 345, Jan. 2025, ISSN: 2076-3417. DOI: 10.3390/app151910345 Accessed: Jan. 24, 2026. [Online]. Available: https://www.mdpi.com/2076-3417/15/19/10345

[59] D. Sklavenitis and D. Kalles, "A scoping review and assessment framework for technical debt in the development and operation of AI/ML competition platforms," *Applied Sciences*, vol. 15, no. 13, p. 7165, Jan. 2025, ISSN: 2076-3417. DOI: 10.3390/app15137165 Accessed: Jan. 24, 2026. [Online]. Available: https://www.mdpi.com/2076-3417/15/13/7165

[60] S. G. Sneha, S. Niha, and D. M. M. Hasan, "Evaluating code clone detection and management: A comprehensive comparison among different techniques and tools along with some effective future directions," in *Proceedings of the 3rd International Conference on Computing Advancements*, ser. ICCA '24, New York, NY, USA: Association for Computing Machinery, Jun. 6, 2025, pp. 207–215, ISBN: 979-8-4007-1382-8. DOI: 10.1145/3723178.3723206 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3723178.3723206

[61] Y. Peng, H.-M. Heyn, and J. Horkoff, *From machine learning documentation to requirements: Bridging processes with requirements languages*, Nov. 19, 2025. DOI: 10.48550/arXiv.2511.15340 arXiv: 2511.15340[cs]. Accessed: Jan. 24, 2026. [Online]. Available: http://arxiv.org/abs/2511.15340

[62] Q. Li et al., "IMPACT: Identifying and classifying multiple sourced and categorized self-admitted technical debts," *ACM Trans. Softw. Eng. Methodol.*, Jul. 10, 2025, Just Accepted, ISSN: 1049-331X. DOI: 10.1145/3747180 Accessed: Jan. 24, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3747180

[63] F. Stagni et al., "The neXt dirac incarnation," *EPJ Web of Conferences*, vol. 337, p. 01 336, 2025, ISSN: 2100-014X. DOI: 10.1051/epjconf/202533701336 Accessed: Jan. 24, 2026. [Online]. Available: https://www.epj-conferences.org/articles/epjconf/abs/2025/22/epjconf_chep2025_01336/epjconf_chep2025_01336.html

[64] J. Perera, E. Tempero, Y.-C. Tu, and K. Blincoe, "Veracity debt: Practitioners voices on managing software requirements concerning veracity," in *Requirements Engineering: Foundation for Software Quality*, A. Hess and A. Susi, Eds., Cham: Springer Nature Switzerland, 2025, pp. 13–28, ISBN: 978-3-031-88531-0. DOI: 10.1007/978-3-031-88531-0_2

[65] Y. Granados Hondares, "Bridging agile software development and modeling," in *Intelligent Information Systems*, L. Pufahl, K. Rosenthal, S. España, and S. Nurcan, Eds., Cham: Springer Nature Switzerland, 2025, pp. 321–328, ISBN: 978-3-031-94590-8. DOI: 10.1007/978-3-031-94590-8_38

[66] J. A. Galindo, J.-M. Horcas, A. Felferning, D. Fernandez-Amoros, and D. Benavides, "FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models," in *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume B*, ser. SPLC '23, vol. B, New York, NY, USA: Association for Computing Machinery, Aug. 28, 2023, pp. 16–19, ISBN: 979-8-4007-0092-7. DOI: 10.1145/3579028.3609008 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3579028.3609008

[67] Y. Li, M. Soliman, and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," *Empirical Software Engineering*, vol. 28, no. 3, 2023, ISSN: 1382-3256. DOI: 10.1007/s10664-023-10297-9

[68] D. OBrien, S. Biswas, S. Imtiaz, R. Abdalkareem, E. Shihab, and H. Rajan, "23 shades of self-admitted technical debt: An empirical study on machine learning software," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022, New York, NY, USA: Association for Computing Machinery, Nov. 9, 2022, pp. 734–746, ISBN: 978-1-4503-9413-0. DOI: 10.1145/3540250.3549088 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3540250.3549088

[69] O. Springer and J. Miler, "A comprehensive overview of software product management challenges," *Empirical Software Engineering*, vol. 27, no. 5, 2022, ISSN: 1382-3256. DOI: 10.1007/s10664-022-10134-5

[70] A. Tornhill and M. Borg, "Code red: The business impact of code quality - a quantitative study of 39 proprietary production codebases," in *Proceedings of the International Conference on Technical Debt*, ser. TechDebt '22, New York, NY, USA: Association for Computing Machinery, Aug. 16, 2022, pp. 11–20, ISBN: 978-1-4503-9304-1. DOI: 10.1145/3524843.3528091 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3524843.3528091

[71] E. A. AlOmar, M. Chouchen, M. W. Mkaouer, and A. Ouni, "Code review practices for refactoring changes: An empirical study on OpenStack," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR '22, New York, NY, USA: Association for Computing Machinery, Oct. 17, 2022, pp. 689–701, ISBN: 978-1-4503-9303-4. DOI: 10.1145/3524842.3527932 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3524842.3527932

[72] R. Yedida and T. Menzies, "How to improve deep learning for software analytics: (a case study with code smell detection)," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ser. MSR '22, New York, NY, USA: Association for Computing Machinery, Oct. 17, 2022, pp. 156–166, ISBN: 978-1-4503-9303-4. DOI: 10.1145/3524842.3528458 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3524842.3528458

[73] Z. Guo et al., "How far have we progressed in identifying self-admitted technical debts? a comprehensive empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 4, 45:1–45:56, Jul. 23, 2021, ISSN: 1049-331X. DOI: 10.1145/3447247 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3447247

[74] B. Vogel-Heuser and F. Bi, "Interdisciplinary effects of technical debt in companies with mechatronic products — a qualitative study," *Journal of Systems and Software*, vol. 171, p. 110 809, Jan. 1, 2021, ISSN: 0164-1212. DOI: 10.1016/j.jss.2020.110809 Accessed: Jan. 23, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121220302119

[75] B. Pérez et al., "Technical debt payment and prevention through the lenses of software architects," *Information and Software Technology*, vol. 140, p. 106 692, Dec. 2021, ISSN: 09505849. DOI: 10.1016/j.infsof.2021.106692 Accessed: Jan. 23, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0950584921001476

[76] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung, "When and how to make breaking changes: Policies and practices in 18 open source software ecosystems," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 4, 42:1–42:56, Jul. 23, 2021, ISSN: 1049-331X. DOI: 10.1145/3447245 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3447245

[77] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "An exploratory study on the introduction and removal of different types of technical debt in deep learning frameworks," *Empirical Software Engineering*, vol. 26, no. 2, p. 16, Mar. 2021, ISSN: 1382-3256, 1573-7616. DOI: 10.1007/s10664-020-09917-5 Accessed: Jan. 23, 2026. [Online]. Available: http://link.springer.com/10.1007/s10664-020-09917-5

[78] X. Wang, J. Liu, L. Li, X. Chen, X. Liu, and H. Wu, "Detecting and explaining self-admitted technical debts with attention-based neural networks," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '20, New York, NY, USA: Association for Computing Machinery, Jan. 27, 2021, pp. 871–

882, ISBN: 978-1-4503-6768-4. DOI: 10.1145/3324884.3416583 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3324884.3416583

[79] J. Liu, Q. Huang, X. Xia, E. Shihab, D. Lo, and S. Li, "Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society*, ser. ICSE-SEIS '20, New York, NY, USA: Association for Computing Machinery, Sep. 18, 2020, pp. 1–10, ISBN: 978-1-4503-7125-4. DOI: 10.1145/3377815.3381377 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3377815.3381377

[80] B. Pérez et al., "What are the practices used by software practitioners on technical debt payment: Results from an international family of surveys," in *Proceedings of the 3rd International Conference on Technical Debt*, ser. TechDebt '20, New York, NY, USA: Association for Computing Machinery, Sep. 25, 2020, pp. 103–112, ISBN: 978-1-4503-7960-1. DOI: 10.1145/3387906.3388632 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3387906.3388632

[81] K. Rindell, K. Bernsmed, and M. G. Jaatun, "Managing security in software: Or: How i learned to stop worrying and manage the security technical debt," in *Proceedings of the 14th International Conference on Availability, Reliability and Security*, ser. ARES '19, New York, NY, USA: Association for Computing Machinery, Aug. 26, 2019, pp. 1–8, ISBN: 978-1-4503-7164-3. DOI: 10.1145/3339252.3340338 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3339252.3340338

[82] E.-M. Arvanitou, A. Ampatzoglou, S. Bibi, A. Chatzigeorgiou, and I. Stamelos, "Monitoring technical debt in an industrial setting," in *Proceedings of the 23rd International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '19, New York, NY, USA: Association for Computing Machinery, Apr. 15, 2019, pp. 123–132, ISBN: 978-1-4503-7145-2. DOI: 10.1145/3319008.3319019 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3319008.3319019

[83] M. Wyrich and J. Bogner, "Towards an autonomous bot for automatic source code refactoring," in *Proceedings of the 1st International Workshop on Bots in Software Engineering*, ser. BotSE '19, Montreal, Quebec, Canada: IEEE Press, May 27, 2019, pp. 24–28. DOI: 10.1109/BotSE.2019.00015 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1109/BotSE.2019.00015

[84] R. Alfayez, P. Behnamghader, K. Srisopha, and B. Boehm, "An exploratory study on the influence of developers in technical debt," in *Proceedings of the 2018 International Conference on Technical Debt*, ser. TechDebt '18, New York, NY, USA: Association for Computing Machinery, May 27, 2018, pp. 1–10, ISBN: 978-1-4503-5713-5. DOI: 10.

1145/3194164.3194165 Accessed: Jan. 23, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3194164.3194165

[85] C. Venters et al., "Software sustainability: Research and practice from a software architecture viewpoint," *Journal of Systems and Software*, vol. 138, Dec. 1, 2017. DOI: 10.1016/j.jss.2017.12.026

ID	Title	Authors	Year	Type	Source
S1	Technical debt is not just technical: An industrial case study in large agile software development	Ahmad et al.	2026	Journal Article	Journal of Systems and Software
S2	Exploring Practitioner's Perception of Conceptual Modelling in Agile Software Development	Granados Hondares et al.	2026	Book Chapter	The Practice of Enterprise Modeling
S3	Leaving the Tech Debt Behind: How to Sustainably Improve the User and Developer Experience of a Legacy Frontend by Designing, Building and Migrating to a New Web Client	Maiwald et al.	2026	Conference Paper	21st International Conference on Web Information Systems and Technologies
S4	Understanding Self-Admitted Technical Debt in Test Code: An Empirical Study	Nakamura et al.	2026	Journal Article	ACM Trans. Softw. Eng. Methodol.
S5	Studying SATD in drone systems with Human-AI collaboration	Rantala et al.	2026	Journal Article	Journal of Systems and Software
S6	A case study on decoding human factors and socio-technical debt in large scale agile project	Herath and Ahmad	2025	Conference Paper	Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion

ID	Title	Authors	Year	Type	Source
S7	A study on reported under-documented non-functional requirements as an indicator of technical debt	Scott et al.	2025	Journal Article	Software Quality Journal
S8	ACE: Automated Technical Debt Remediation with Validated Large Language Model Refactorings	Tornhill et al.	2025	Conference Paper	Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (Companion)
S9	Adapt and Overcome - How Agile Practitioners Adapt to Issues that Impede the Delivery of Value: An Interview Study	Meckenstock and Wallmichrath	2025	Book Chapter	Agile Processes in Software Engineering and Extreme Programming
S10	An Empirical Study of Self-Admitted Technical Debt in Machine Learning Software	Bhatia et al.	2025	Journal Article	ACM Trans. Softw. Eng. Methodol.
S11	Branch Drift: A Visually Explainable Metric for Consistency Monitoring in Collaborative Software Development	Kegel et al.	2025	Journal Article	IEEE Access
S12	Microservices-Driven Automation in Full-Stack Development: Bridging Efficiency and Innovation With FSMicroGenerator	Khalfaoui et al.	2025	Journal Article	IEEE Access

ID	Title	Authors	Year	Type	Source
S13	PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects	Aljohani and Do	2025	Conference Paper	Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering
S14	Product Guardian Role and Socio-Technical Debt Management in Large-Scale Agile	Herath et al.	2025	Conference Paper	Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion
S15	QUPER-MAn: Benchmark-Guided Target Setting for Maintainability Requirements	Borg et al.	2025	Workshop Paper	Proceedings of the 1st International Workshop on Responsible Software Engineering
S16	Requirements Technical Debt Through the Lens of Environment Assumptions	Alenazi	2025	Conference Paper	2025 IEEE/ACM International Conference on Technical Debt (TechDebt)
S17	Tracing Code Quality Evolution: Insights from a Data Structures and Algorithms Course	Prajapati and Acuña	2025	Conference Paper	2025 IEEE Frontiers in Education Conference (FIE)
S18	Understanding practitioners' reasoning and requirements for efficient tool support in technical debt management	Biazotto et al.	2025	Journal Article	Empirical Software Engineering
S19	Uncovering the Implicit: A Comparative Evaluation of Modeling Approaches for Environmental Assumptions	Alenazi	2025	Journal Article	Applied Sciences

ID	Title	Authors	Year	Type	Source
S20	The Role of Environmental Assumptions in Shaping Requirements Technical Debt	Alenazi	2025	Journal Article	Applied Sciences
S21	A Scoping Review and Assessment Framework for Technical Debt in the Development and Operation of AI/ML Competition Platforms	Sklavenitis and Kalles	2025	Journal Article	Applied Sciences
S22	Evaluating Code Clone Detection and Management: A Comprehensive Comparison among different Techniques and Tools along with some Effective Future Directions	Sneha et al.	2025	Conference Paper	Proceedings of the 3rd International Conference on Computing Advancements
S23	From Machine Learning Documentation to Requirements: Bridging Processes with Requirements Languages	Peng et al.	2025	Preprint	arXiv
S24	IMPACT: Identifying and Classifying Multiple Sourced and Categorized Self-Admitted Technical Debts	Li et al.	2025	Journal Article	ACM Trans. Softw. Eng. Methodol.
S25	The neXt Dirac incarnation	Stagni et al.	2025	Journal Article	EPJ Web of Conferences

ID	Title	Authors	Year	Type	Source
S26	Veracity Debt: Practitioners Voices on Managing Software Requirements Concerning Veracity	Perera et al.	2025	Book Chapter	Requirements Engineering: Foundation for Software Quality
S27	Bridging Agile Software Development and Modeling	Granados Hondares	2025	Book Chapter	Intelligent Information Systems
S28	FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models	Galindo et al.	2023	Conference Paper	Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume B
S29	Automatic identification of self-admitted technical debt from four different sources	Li et al.	2023	Journal Article	Empirical Software Engineering
S30	Sustainable software engineering: Reflections on advances in research and practice	Venters et al.	2023	Journal Article	Information and Software Technology
S31	23 shades of self-admitted technical debt: an empirical study on machine learning software	OBrien et al.	2022	Conference Paper	Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering
S32	Agile MERODE: a model-driven software engineering method for user-centric and value-based development	Snoeck and Wautelet	2022	Journal Article	Software and Systems Modeling

ID	Title	Authors	Year	Type	Source
S33	Identification and measurement of Requirements Technical Debt in software development: A systematic literature review	Melo et al.	2022	Journal Article	Journal of Systems and Software
S34	A comprehensive overview of software product management challenges	Springer and Miler	2022	Journal Article	Empirical Software Engineering
S35	Code red: the business impact of code quality - a quantitative study of 39 proprietary production codebases	Tornhill and Borg	2022	Conference Paper	Proceedings of the International Conference on Technical Debt
S36	Code review practices for refactoring changes: an empirical study on OpenStack	AlOmar et al.	2022	Conference Paper	Proceedings of the 19th International Conference on Mining Software Repositories
S37	How to improve deep learning for software analytics: (a case study with code smell detection)	Yedida and Menzies	2022	Conference Paper	Proceedings of the 19th International Conference on Mining Software Repositories
S38	Towards optimal quality requirement documentation in agile software development: A multiple case study	Behutiye et al.	2022	Journal Article	Journal of Systems and Software
S39	How Far Have We Progressed in Identifying Self-admitted Technical Debts? A Comprehensive Empirical Study	Guo et al.	2021	Journal Article	ACM Trans. Softw. Eng. Methodol.

ID	Title	Authors	Year	Type	Source
S40	Interdisciplinary effects of technical debt in companies with mechatronic products — a qualitative study	Vogel-Heuser and Bi	2021	Journal Article	Journal of Systems and Software
S41	Technical debt payment and prevention through the lenses of software architects	Pérez et al.	2021	Journal Article	Information and Software Technology
S42	When and How to Make Breaking Changes: Policies and Practices in 18 Open Source Software Ecosystems	Bogart et al.	2021	Journal Article	ACM Trans. Softw. Eng. Methodol.
S43	An exploratory study on the introduction and removal of different types of technical debt in deep learning frameworks	Liu et al.	2021	Journal Article	Empirical Software Engineering
S44	Detecting and explaining self-admitted technical debts with attention-based neural networks	Wang et al.	2021	Conference Paper	Proceedings of the IEEE/ACM International Conference on Automated Software Engineering
S45	Actions and impediments for technical debt prevention: results from a global family of industrial surveys	Freire et al.	2020	Conference Paper	Proceedings of the 35th Annual ACM Symposium on Applied Computing
S46	Experiences with technical debt and management strategies in production systems engineering	Waltersdorfer et al.	2020	Conference Paper	Proceedings of the 3rd International Conference on Technical Debt

ID	Title	Authors	Year	Type	Source
S47	Feature dependencies in automotive software systems: Extent, awareness, and refactoring	Vogelsang	2020	Journal Article	Journal of Systems and Software
S48	Fostering continuous value proposition innovation through freelancer involvement in software startups: Insights from multiple case studies	Gupta et al.	2020	Journal Article	Sustainability (Switzerland)
S49	Hearing the Voice of Software Practitioners on Causes, Effects, and Practices to Deal with Documentation Debt	Rios et al.	2020	Book	N/A
S50	Identification and Remediation of Self-Admitted Technical Debt in Issue Trackers	Li et al.	2020	Conference Paper	SEAA (IEEE)
S51	Identifying self-admitted technical debt through code comment analysis with a contextualized vocabulary	Farias et al.	2020	Journal Article	Information and Software Technology
S52	Is using deep learning frameworks free? characterizing technical debt in deep learning frameworks	Liu et al.	2020	Conference Paper	Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society

ID	Title	Authors	Year	Type	Source
S53	What are the practices used by software practitioners on technical debt payment: results from an international family of surveys	Pérez et al.	2020	Conference Paper	Proceedings of the 3rd International Conference on Technical Debt
S54	A Systematic Mapping Study on Security in Agile Requirements Engineering	Villamizar et al.	2018	Conference Paper	SEAA (IEEE)
S55	Causes and Effects of the Presence of Technical Debt in Agile Software Projects	Rios et al.	2019	Conference Paper	N/A
S56	Managing Security in Software: Or: How I Learned to Stop Worrying and Manage the Security Technical Debt	Rindell et al.	2019	Conference Paper	Proceedings of the 14th International Conference on Availability, Reliability and Security
S57	Monitoring Technical Debt in an Industrial Setting	Arvanitou et al.	2019	Conference Paper	Proceedings of the 23rd International Conference on Evaluation and Assessment in Software Engineering
S58	Requirements Quality Is Quality in Use	Femmer and Vogelsang	2019	Journal Article	IEEE Software
S59	Technical Debt Triage in Backlog Management	Besker et al.	2019	Conference Paper	2019 IEEE/ACM International Conference on Technical Debt (TechDebt)

ID	Title	Authors	Year	Type	Source
S60	Towards a Holistic Definition of Requirements Debt	Lenarduzzi and Fucci	2019	Conference Paper	2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)
S61	Towards an autonomous bot for automatic source code refactoring	Wyrich and Bogner	2019	Workshop Paper	Proceedings of the 1st International Workshop on Bots in Software Engineering
S62	Automatic Classifying Self-Admitted Technical Debt Using N-Gram IDF	Wattanakriengkrai et al.	2019	Conference Paper	Proc. Asia Pac. Softw. Eng. Conf. (APSEC)
S63	Management of quality requirements in agile and rapid software development: A systematic mapping study	Behutiye et al.	2019	Journal Article	Information and Software Technology
S64	An exploratory study on the influence of developers in technical debt	Alfayez et al.	2018	Conference Paper	Proceedings of the 2018 International Conference on Technical Debt
S65	Crowd-Focused Semi-Automated Requirements Engineering for Evolution Towards Sustainability	Seyff et al.	2018	Conference Paper	IEEE International Requirements Engineering Conference (RE)
S66	Exploration of technical debt in start-ups	Klotins et al.	2018	Conference Paper	Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice

ID	Title	Authors	Year	Type	Source
S67	Identifying Design and Requirement Self-Admitted Technical Debt Using N-gram IDF	Wattanakriengkrai et al.	2018	Conference Paper	IWESEP (IEEE)
S68	The Evolution of Requirements Practices in Software Startups	Gralha et al.	2018	Conference Paper	2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)
S69	Software Sustainability: Research and Practice from a Software Architecture Viewpoint	Venters et al.	2017	Journal Article	Journal of Systems and Software

ID	Title	Author / Organization	Year	Type	Source
G1	What is Tech Debt? Signs & How to Effectively Manage It | Atlassian	Atlassian	n.d.	Online Source	https://www.atlassian.com/agile/software-development/technical-debt
G2	What is Technical Debt in Software Development and How to Manage It?	GeeksforGeeks	n.d.	Online Source	https://www.geeksforgeeks.org/blogs/what-is-technical-debt-in-software-development-and-how-to-manage-it/
G3	What Is Technical Debt: A Guide	Staff	2024	Online Source	https://www.coursera.org/articles/technical-debt
G4	What is technical debt?	GitHub	2024	Online Source	https://github.com/resources/articles/what-is-technical-debt
G5	Technical Debt (Tech Debt): A Complete Guide	Confluent	n.d.	Online Source	https://www.confluent.io/learn/tech-debt/
G6	What is Technical Debt? Examples, Prevention & Best Practices	Mendix	2024	Online Source	https://www.mendix.com/blog/what-is-technical-debt/
G7	Technical Debt | Definition & Guide	Sonarsource	n.d.	Online Source	https://www.sonarsource.com/resources/library/technical-debt/
G8	Technical Debt: How to Identify, Measure, and Manage	Islam	2024	Online Source	https://innovirtual.de/technical-debt-how-to-identify-measure-and-manage/
G9	How to Measure Technical Debt in Software Development	Jain and Solutions	2025	Online Source	https://visuresolutions.com/alm-guide/measure-technical-debt/
G10	Technical Debt & How To Manage It	Mitton	2023	Online Source	`https://www.splunk.com/en_us/blog/learn/technical-debt.html`